

Project 2 Writeup

Algorithms

The algorithm used is the penalty method. The outer constrained problem is converted into a sequence of unconstrained problems with the objective

$$\phi(x) = f(x) + \rho_1 \sum_i \mathbf{1}\{c_i(x) > 0\} + \rho_2 \sum_i \max(c_i(x), 0)^2$$

The inner unconstrained problem is solved with normalized gradient descent and backtracking line search. The gradient used by the inner method is

$$\nabla \phi(x) \approx \nabla f(x) + 2\rho_2 J_c(x)^T \max(c(x), 0)$$

where the constraint Jacobian $J_c(x)$ is estimated by forward finite differences. The count penalty is included in the objective value, but not in the gradient because it is piecewise constant and has zero gradient almost everywhere.

For all five problems I used the same method with $\rho_1 = 1$, $\rho_2 = 10$, and $\gamma = 4$, where both penalty weights are multiplied by γ after each outer penalty iteration. The inner minimization is capped at five accepted descent steps. This cap was chosen empirically: it gives the outer loop frequent chances to increase the penalty weights instead of spending the whole evaluation budget optimizing a weak early penalty objective.

Why it works. Violated constraints add a positive penalty, so descent on ϕ pushes iterates toward feasibility. Increasing ρ_1 and ρ_2 makes feasibility more important over time. The finite-difference Jacobian gives the method a local direction for reducing constraint violation without hard-coding any constraint gradients.

Pros. The method is generic, uses only f , g , and c , and works for the simple problems as well as higher-dimensional secret problems because the constraints are flattened before computing violations.

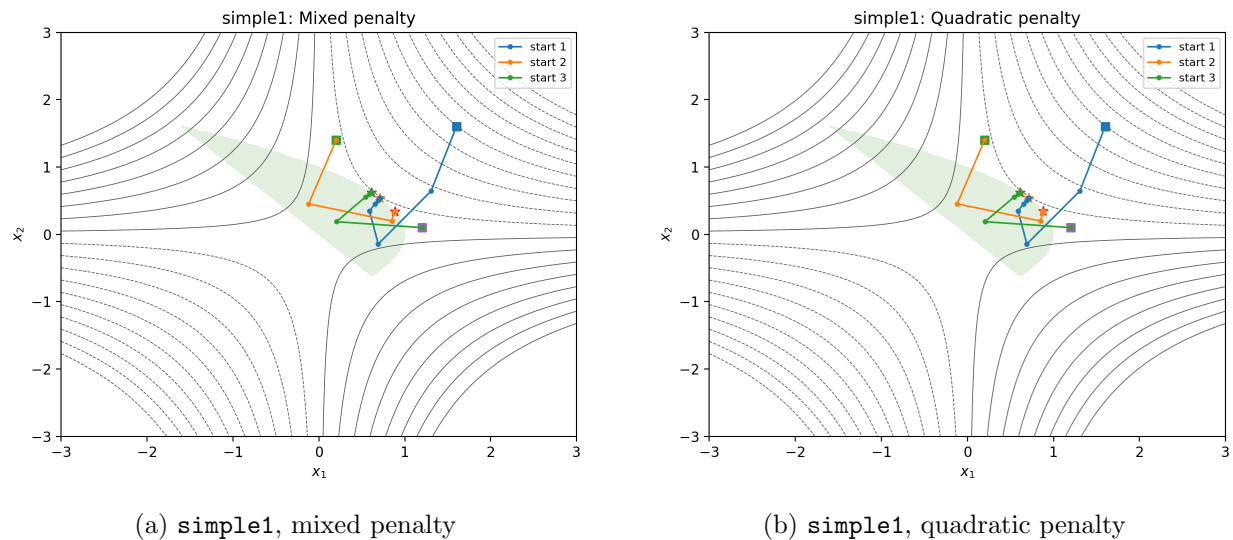
Cons. The method spends extra evaluations estimating $J_c(x)$. It is also sensitive to penalty scaling: if the penalty is too weak, feasibility improves slowly; if it is too strong, the penalty term can dominate the original objective.

For comparison plots, I used a pure quadratic penalty

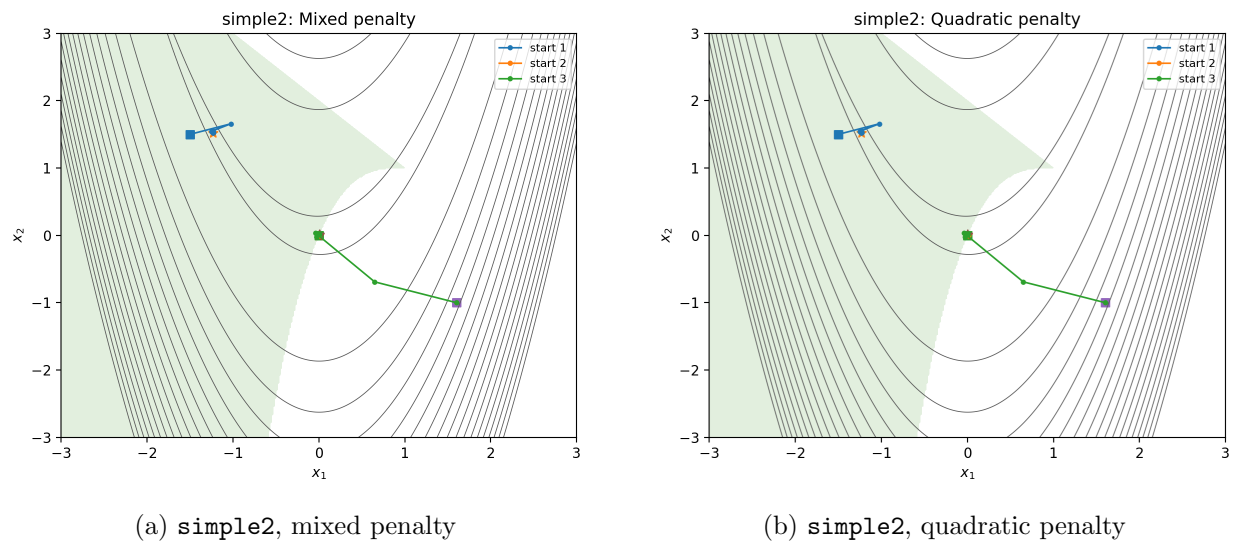
$$\phi_q(x) = f(x) + \rho_2 \sum_i \max(c_i(x), 0)^2$$

Feasible Regions and Paths

For `simple1`, both methods move the iterates from initially infeasible or weakly feasible points toward the green feasible set. The mixed penalty tends to push more directly toward feasibility because the count penalty changes the line search objective whenever a constraint remains violated.

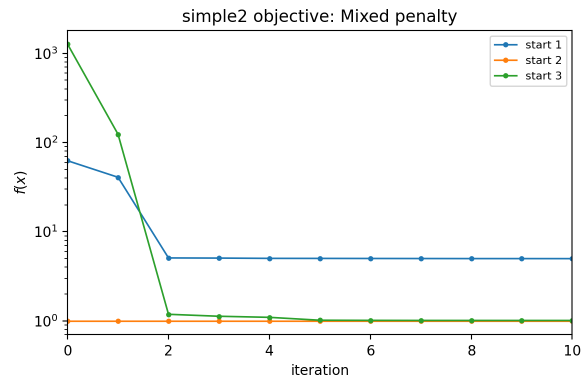
Figure 1: Comparison of two penalty methods on **simple1**.

For **simple2**, the feasible region contains the constrained function minimum near $(1,1)$. The paths show the tradeoff between following the curved function valley and reducing constraint violation; the penalty weights increasingly favor feasibility as the outer iterations continue. The paths are still visually similar because both methods use the same normalized descent direction, and the count penalty does not contribute a useful smooth gradient.

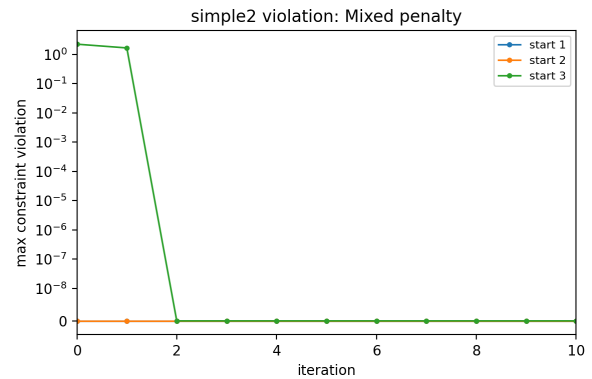
Figure 2: Comparison of two penalty methods on **simple2**.

Convergence and Violation

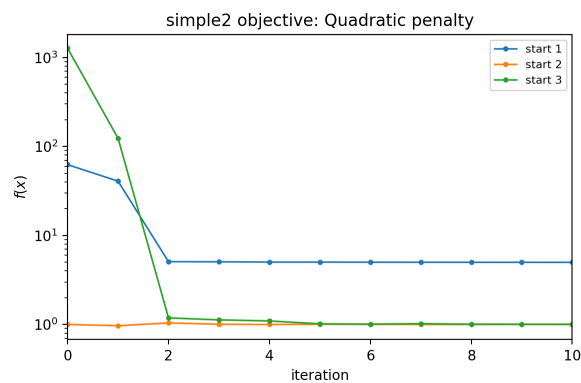
As plots indicate, the mixed penalty reduces the maximum violation more steadily. The pure quadratic penalty is more jagged on **simple2**, meaning some accepted steps move away from feasibility before recovering. For this problem, that makes the quadratic penalty less reliable than the mixed penalty.



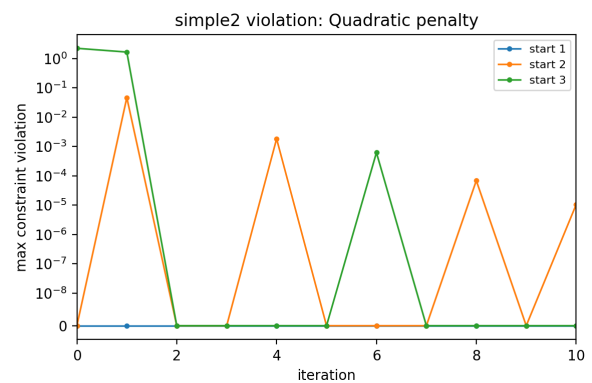
(a) Mixed penalty objective values



(b) Mixed penalty violation

Figure 3: `simple2` performance for the mixed penalty method.

(a) Quadratic penalty objective values



(b) Quadratic penalty violation

Figure 4: `simple2` performance for the pure quadratic penalty method.

Code

```

1 def c_flat(c, x):
2     return np.asarray(c(x), dtype=float).reshape(-1)
3
4 def descent_step(f, g, x, n, count):
5     f_x = f(x)
6     g_x = g(x)
7     g_norm = np.linalg.norm(g_x)
8     d = -g_x / g_norm
9
10    alpha = 1.0
11    p = 0.5
12    beta = 1e-4
13
14    while count() < n:
15        x_trial = x + alpha * d
16        f_trial = f(x_trial)

```

```
17         if f_trial <= f_x + beta * alpha * (g_x.T @ d):
18             return x_trial, True
19         alpha *= p
20     return x, False
21
22 def objective(x, f, c, rho1, rho2, n, count):
23     if count() + 2 > n:
24         return np.inf
25     c_x = c_flat(c, x)
26     p_count = np.sum(c_x > 0.0)
27     p_quad = np.sum(np.maximum(c_x, 0.0) ** 2)
28     return f(x) + rho1 * p_count + rho2 * p_quad
29
30 def grad_objective(x, f, g, c, rho2, n, count):
31     if count() + 3 + len(x) > n:
32         return None
33
34     g_x = g(x)
35     c_x = c_flat(c, x)
36     m = np.maximum(c_x, 0.0)
37
38     eps = 1e-6
39     grad_c = np.zeros((len(c_x), len(x)))
40     for j in range(len(x)):
41         x_eps = x.copy()
42         x_eps[j] += eps
43         grad_c[:, j] = (c_flat(c, x_eps) - c_x) / eps
44
45     return g_x + 2.0 * rho2 * (grad_c.T @ m)
46
47 def penalty_method(x0, f, g, c, n, count, rho1=1.0, rho2=10.0, gamma=4.0):
48     x = x0.copy()
49     best_x = x.copy()
50     best_violation = np.inf
51
52     while count() + 8 + len(x) <= n:
53         x, _, _, _ = minimize(
54             x,
55             lambda z: objective(z, f, c, rho1, rho2, n, count),
56             lambda z: grad_objective(z, f, g, c, rho2, n, count),
57             n,
58             count,
59             max_steps=5,
60         )
61
62         c_x = c_flat(c, x)
63         max_violation = np.max(np.maximum(c_x, 0.0))
64         if max_violation < best_violation:
65             best_x = x.copy()
66             best_violation = max_violation
67         if max_violation <= 0.0:
68             return x
69
70     rho1 *= gamma
```

```
71         rho2 *= gamma
72
73     return best_x
```